



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 2	Tema: Fundamentos de Arquitectura Backend
	Semana 3	Tutor: Ing. Victoria Castro

Introducción:

El desarrollo de aplicaciones web modernas exige una comprensión sólida de los principios arquitectónicos que sustentan el backend. Más allá de la implementación técnica, el diseño de sistemas robustos requiere adoptar modelos estructurales que garanticen escalabilidad, mantenibilidad, seguridad y evolución tecnológica sostenible. El objetivo del presente documento es formar criterio arquitectónico, no únicamente habilidades técnicas.

Sección 1:

Fundamentos de Arquitectura Backend y Spring Boot 3.x

El backend constituye la capa lógica de una aplicación web responsable de la gestión de reglas de negocio, acceso a datos, seguridad, validaciones y exposición de servicios.

En el ecosistema Java moderno, Spring Boot 3.x se ha consolidado como un estándar para la construcción de aplicaciones backend empresariales. Su propuesta se basa en:

- Configuración automática (auto-configuration).
- Convención sobre configuración.
- Integración modular del ecosistema Spring.
- Enfoque en aplicaciones productivas desde el inicio.



Inicialización de Proyecto

La inicialización de un proyecto en Spring Boot no es únicamente un proceso técnico, sino una decisión arquitectónica inicial que define:

- Tipo de empaquetado (JAR/WAR).
- Dependencias fundamentales (Web, Data JPA, Security).
- Sistema de construcción (Maven o Gradle).
- Versión de Java.

El uso de generadores oficiales como Spring Initializr permite establecer una estructura estandarizada que promueve buenas prácticas desde el inicio del ciclo de vida del software.

Gestión de Dependencias con Maven

Maven es una herramienta de automatización de construcción y gestión de dependencias basada en el modelo POM (Project Object Model).



Su relevancia estratégica radica en:

- Declaratividad de dependencias.
- Resolución automática de librerías.
- Gestión de versiones coherentes.
- Integración con repositorios centrales (Maven Central).

En el contexto de Spring Boot, Maven facilita:

- Control de versiones del framework.
- Inclusión modular de starters.
- Integración con pipelines CI/CD.

La correcta gestión de dependencias impacta directamente en la estabilidad, seguridad y mantenibilidad del proyecto.

Sección 2:

Arquitectura Cliente-Servidor Desacoplada

El modelo Cliente-Servidor establece una separación clara entre:

- **Cliente:** interfaz de usuario (Frontend).
- **Servidor:** lógica de negocio y acceso a datos (Backend).

En arquitecturas modernas, esta relación se implementa de forma desacoplada mediante APIs REST o servicios HTTP, permitiendo que:

- El frontend puede desarrollarse con frameworks independientes (React, Angular, Vue).
- El backend puede evolucionar sin afectar la interfaz.
- Se habilite interoperabilidad con aplicaciones móviles o sistemas externos.
- El desacoplamiento reduce dependencias rígidas y favorece la escalabilidad horizontal.

Sección 3:

Modelo de N-Capas

En este modelo, la comunicación es **lineal y descendente**. Cada capa tiene una responsabilidad única.

- **Capa de Presentación (Controllers):** No debe contener lógica de negocio. Su única función es recibir la petición HTTP, validar que los datos de entrada sean correctos y pasar la estafeta a la siguiente capa.
- **Capa de Negocio (Services):** Es el "cerebro" del sistema. Aquí es donde se aplican las reglas que mencionas en tu texto. **Mejora:** Asegúrate de que los servicios no conozcan detalles de SQL o de la web.
- **Capa de Persistencia (Repositories):** Se encarga de la comunicación con el exterior (Base de Datos).
- **Base de Datos:** El almacenamiento físico de la información.

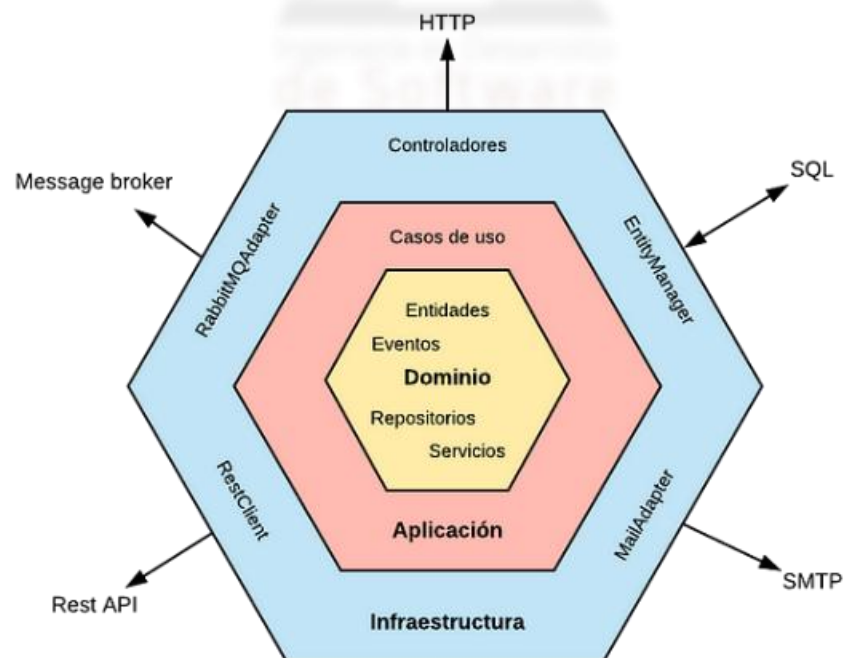
Modelo N-Capas



El mayor riesgo en las N-Capas es el "Acoplamiento Transmitido", donde un cambio en la base de datos termina obligándote a cambiar el código hasta en el Controlador.

Sección 4:

Evolución a Arquitectura Hexagonal



La Arquitectura Hexagonal, propuesta por Alistair Cockburn, plantea un modelo orientado a aislar el núcleo del dominio del resto de dependencias externas.

Su principio central establece que:

"El dominio no debe depender de frameworks, bases de datos o interfaces externas."

Componentes conceptuales:

- Núcleo del dominio (modelo y reglas de negocio).
- Puertos (interfaces).
- Adaptadores (implementaciones externas).

A diferencia del modelo N-Capas tradicional, la Arquitectura Hexagonal invierte la dependencia, permitiendo que:

- La base de datos puede cambiar sin afectar el dominio.
- El framework puede reemplazarse sin alterar la lógica central.
- Se facilita el testing unitario independiente.

Principios Fundamentales

- Inversión de dependencias.
- Aislamiento del dominio.
- Flexibilidad tecnológica.
- Orientación a largo plazo.

Beneficios

- Mayor resiliencia ante cambios tecnológicos.
- Facilita migraciones arquitectónicas.
- Permite transicionar hacia microservicios.
- Reduce deuda técnica acumulada.

La Arquitectura Hexagonal no reemplaza el modelo N-Capas, sino que lo evoluciona hacia un enfoque más orientado al dominio.

Sección 5:

Integración Conceptual: Spring Boot, Maven y Arquitectura

El uso de Spring Boot 3.x dentro de una arquitectura N-Capas o Hexagonal no es una decisión meramente técnica, sino estratégica.

Spring Boot provee:

- Infraestructura.
- Configuración.
- Integración con bases de datos.
- Gestión de dependencias mediante Maven.

La arquitectura define:

- Cómo se organiza el código.
- Cómo fluyen las dependencias.
- Cómo se mantiene la coherencia estructural.

La combinación adecuada de herramientas y principios arquitectónicos permite construir aplicaciones:

- Escalables.
- Seguras.
- Testables.
- Evolutivas.

El backend moderno no se trata únicamente de escribir controladores, sino de diseñar sistemas sostenibles en el tiempo.

Actividad

¡Hola a todos! Tras revisar nuestro material de lectura sobre **Fundamentos de Arquitectura Backend**, es momento de reflexionar sobre las herramientas que sostienen la ingeniería de software actual.

Te invito a participar en nuestro **Foro de Discusión de la Unidad 2** y responder con tus palabras la siguiente pregunta:

¿Cómo cambiaría la mantenibilidad de un proyecto backend si la lógica de negocio dependiera directamente de la base de datos sin una capa de servicios intermedia?

Bibliografía

1. Johnson, R., Hoeller, J., Arendsen, A., Risberg, T., & Sampaleanu, C. (2023). *Spring Framework Documentation*. <https://docs.spring.io/spring-framework/reference/>
2. Pivotal Software. (2023). *Spring Boot Reference Documentation*. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
3. Sonatype. (2023). *Maven: The Complete Reference*. <https://maven.apache.org/guides/>
4. Cockburn, A. (2005). *Hexagonal Architecture*.
5. Spring.io (2019). *Spring.io*. <https://start.spring.io/>